Análisis Multivariado de Varianza

Ejemplo: Comparación de Métodos de Enseñanza

Actividad MANOVA

Dr. Hugo García Tecocoatzi

```
# Bibliotecas necesarias
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from scipy import stats
from sklearn.datasets import load_iris, load_wine, load_diabetes # Necesa
from scipy.stats import f_oneway
from statsmodels.multivariate.manova import MANOVA
from statsmodels.stats.multicomp import pairwise_tukeyhsd

import statsmodels.api as sm
from statsmodels.formula.api import ols

%matplotlib inline
```

```
# Generar datos sintéticos para el ejemplo
np.random.seed(42)

# Parámetros
n_por_grupo = 30
grupos = ['A', 'B', 'C']

# Método A: Enseñanza tradicional
# Matemáticas: media 75, Lectura: media 70
mat_A = np.random.normal(75, 8, n_por_grupo)
lec_A = np.random.normal(70, 8, n_por_grupo)

# Método B: Enseñanza activa
# Matemáticas: media 82, Lectura: media 78
mat_B = np.random.normal(82, 8, n_por_grupo)
lec_B = np.random.normal(78, 8, n_por_grupo)

# Método C: Enseñanza híbrida
# Matemáticas: media 78, Lectura: media 75
mat_C = np.random.normal(78, 8, n_por_grupo)
lec_C = np.random.normal(75, 8, n_por_grupo)
```

```
# Crear DataFrame
df = pd.DataFrame({
    'Metodo': ['A']*n_por_grupo + ['B']*n_por_grupo + ['C']*n_por_grupo,
    'Matematicas': np.concatenate([mat_A, mat_B, mat_C]),
    'Lectura': np.concatenate([lec_A, lec_B, lec_C])
})

# Asegurar que las calificaciones estén en rango [0, 100]
df['Matematicas'] = np.clip(df['Matematicas'], 0, 100)
df['Lectura'] = np.clip(df['Lectura'], 0, 100)

print("Datos generados exitosamente")
print(f"Forma del dataset: {df.shape}")
print(f"Grupos: {df['Metodo'].unique()}")
df.head(10)
```

```
Datos generados exitosamente
Forma del dataset: (90, 3)
Grupos: ['A' 'B' 'C']
```

	Metodo	Matematicas	Lectura
0	A	78.973713	65.186347
1	A	73.893886	84.818225
2	A	80.181508	69.892022
3	A	87.184239	61.538313
4	A	73.126773	76.580359
5	A	73.126904	60.233251
6	A	87.633703	71.670909
7	A	81.139478	54.322639
8	A	71.244205	59.374512
9	A	79.340480	71.574890

Pasos siguientes: Generar código con df | New interactive sheet

```
#  Estadísticas descriptivas por grupo
print("ESTADÍSTICAS DESCRIPTIVAS POR GRUPO: \n")

estadisticas = df.groupby('Metodo').agg({
    'Matematicas': ['mean', 'std', 'min', 'max'],
    'Lectura': ['mean', 'std', 'min', 'max']
}).round(2)

print(estadisticas)
```

```
ESTADÍSTICAS DESCRIPTIVAS POR GRUPO:

       Matematicas                      Lectura
```

```
                mean   std    min    max    mean   std    min    max
Metodo
A              73.49  7.20  59.69  87.63   69.03  7.45  54.32  84.82
B              82.10  7.94  61.04  94.52   77.84  7.27  62.65  97.71
C              77.24  8.19  65.14  95.52   77.23  7.58  65.47  96.76
```

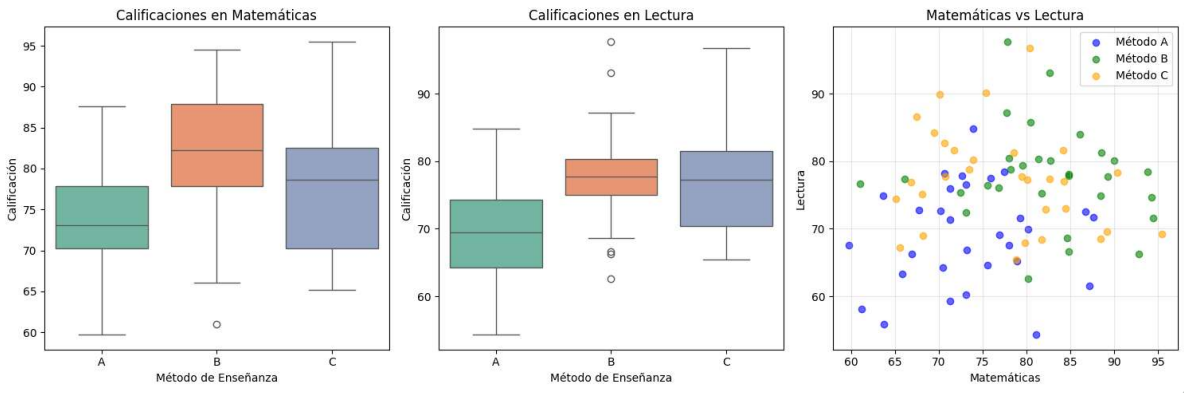
```
# Visualización
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Boxplot Matemáticas
sns.boxplot(x='Metodo', y='Matematicas', data=df, ax=axes[0], palette='Set2'
axes[0].set_title('Calificaciones en Matemáticas')
axes[0].set_xlabel('Método de Enseñanza')
axes[0].set_ylabel('Calificación')

# Boxplot Lectura
sns.boxplot(x='Metodo', y='Lectura', data=df, ax=axes[1], palette='Set2')
axes[1].set_title('Calificaciones en Lectura')
axes[1].set_xlabel('Método de Enseñanza')
axes[1].set_ylabel('Calificación')

# Scatter plot Matemáticas vs Lectura
colors = {'A': 'blue', 'B': 'green', 'C': 'orange'}
for metodo in df['Metodo'].unique():
    subset = df[df['Metodo'] == metodo]
    axes[2].scatter(subset['Matematicas'], subset['Lectura'],
                    alpha=0.6, label=f'Método {metodo}', color=colors[metodo
axes[2].set_xlabel('Matemáticas')
axes[2].set_ylabel('Lectura')
axes[2].set_title('Matemáticas vs Lectura')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



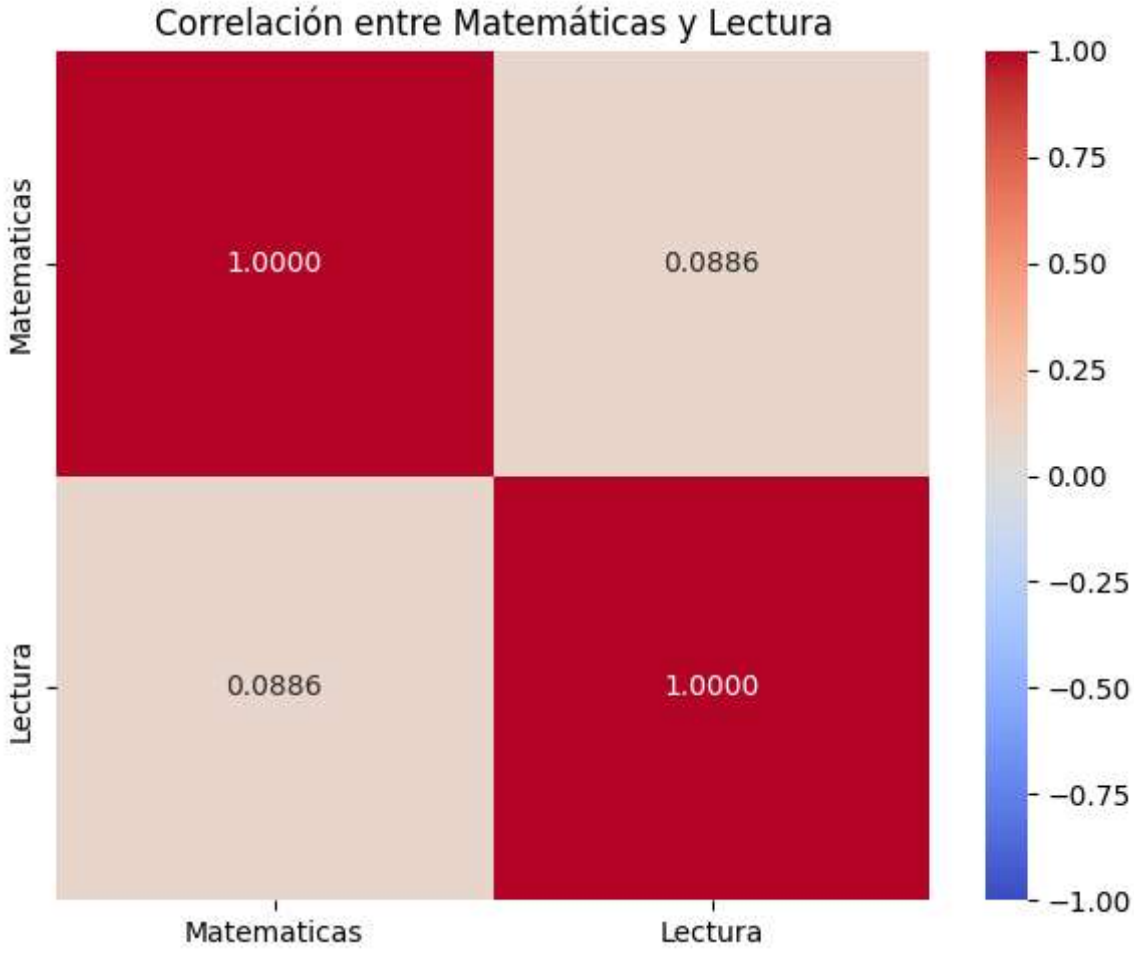

```
# Matriz de correlación
print("MATRIZ DE CORRELACIÓN\n")
print(df[['Matematicas', 'Lectura']].corr().round(4))

# Visualización
plt.figure(figsize=(6, 5))
sns.heatmap(df[['Matematicas', 'Lectura']].corr(),
```

```
            annot=True, cmap='coolwarm', center=0,
            vmin=-1, vmax=1, fmt='.4f')
plt.title('Correlación entre Matemáticas y Lectura')
plt.tight_layout()
plt.show()
```

```
MATRIZ DE CORRELACIÓN

             Matematicas  Lectura
Matematicas       1.0000   0.0886
Lectura           0.0886   1.0000
```




```
#  MANOVA - Prueba principal
print("MANOVA \n")

# Ajustar MANOVA
manova = MANOVA.from_formula('Matematicas + Lectura ~ Metodo', data=df)
resultado = manova.mv_test()

print(resultado)
```

```
MANOVA

                  Multivariate linear model
============================================================

------------------------------------------------------------
       Intercept         Value  Num DF  Den DF  F Value  Pr > F
------------------------------------------------------------
          Wilks' lambda  0.0146 2.0000 86.0000 2907.3547 0.0000
```

```
         Pillai's trace  0.9854 2.0000 86.0000 2907.3547 0.0000
 Hotelling-Lawley trace 67.6129 2.0000 86.0000 2907.3547 0.0000
    Roy's greatest root 67.6129 2.0000 86.0000 2907.3547 0.0000
---------------------------------------------------------------


---------------------------------------------------------------
           Metodo         Value  Num DF  Den DF  F Value Pr > F
---------------------------------------------------------------
            Wilks' lambda 0.6317 4.0000 172.0000 11.1040 0.0000
           Pillai's trace 0.3790 4.0000 174.0000 10.1715 0.0000
   Hotelling-Lawley trace 0.5662 4.0000 102.1690 12.1286 0.0000
      Roy's greatest root 0.5346 2.0000  87.0000 23.2562 0.0000
===============================================================
```

```
#  Extraer resultados de los cuatro estadísticos
print("=== RESULTADOS DE LOS CUATRO ESTADÍSTICOS ===\n")

# Extraer resultados del MANOVA
resultados_mv = manova.mv_test().results['Metodo']['stat']

print("Estadísticos de prueba:")
print(resultados_mv.round(4))
```

```
=== RESULTADOS DE LOS CUATRO ESTADÍSTICOS ===

Estadísticos de prueba:
                          Value Num DF      Den DF    F Value Pr > F
Wilks' lambda          0.631651      4       172.0  11.104044    0.0
Pillai's trace         0.379027    4.0       174.0  10.171469    0.0
Hotelling-Lawley trace 0.566246      4  102.169014  12.128592    0.0
Roy's greatest root    0.534625      2          87  23.256178    0.0
```

```
# Interpretación de resultados
print("=== INTERPRETACIÓN DE RESULTADOS ===\n")

# Verificar significancia de cada estadístico
for estadistico in resultados_mv.index:
    if 'p-value' in resultados_mv.columns:
        p_valor = resultados_mv.loc[estadistico, 'Pr > F']
        valor = resultados_mv.loc[estadistico, 'Value']

        if p_valor < 0.05:
            print(f"✅ {estadistico}: Valor = {valor:.4f}, p = {p_valor:.4f}
        else:
            print(f"❌ {estadistico}: Valor = {valor:.4f}, p = {p_valor:.4f}

print("\n Conclusión:")
print("Si todos los estadísticos son significativos (p < 0.05),")
print("existe evidencia de que los métodos de enseñanza difieren")
print("en el patrón conjunto de Matemáticas y Lectura.")
```

```
=== INTERPRETACIÓN DE RESULTADOS ===


 Conclusión:
```

```
Si todos los estadísticos son significativos (p < 0.05),
existe evidencia de que los métodos de enseñanza difieren
en el patrón conjunto de Matemáticas y Lectura.
```

```
# ANOVA univariado post-hoc (con corrección de Bonferroni)
print("=== ANOVA UNIVARIADO POST-HOC ===\n")

# ANOVA para Matemáticas
print("ANOVA para Matemáticas:")
modelo_mat = ols('Matematicas ~ Metodo', data=df).fit()
anova_mat = sm.stats.anova_lm(modelo_mat, typ=2)
print(anova_mat.round(4))

# ANOVA para Lectura
print("\n ANOVA para Lectura:")
modelo_lec = ols('Lectura ~ Metodo', data=df).fit()
anova_lec = sm.stats.anova_lm(modelo_lec, typ=2)
print(anova_lec.round(4))

# Corrección de Bonferroni
alpha = 0.05
alpha_bonferroni = alpha / 2  # 2 variables dependientes

print(f"\n Corrección de Bonferroni:")
print(f"   α original = {alpha}")
print(f"   α ajustado = {alpha_bonferroni}")

# Interpretación
p_mat = anova_mat.loc['Metodo', 'PR(>F)']
p_lec = anova_lec.loc['Metodo', 'PR(>F)']

print(f"\n   Matemáticas: p = {p_mat:.4f} → {'Significativo' if p_mat < alph
print(f"   Lectura: p = {p_lec:.4f} → {'Significativo' if p_lec < alpha_bonf
```

```
=== ANOVA UNIVARIADO POST-HOC ===

ANOVA para Matemáticas:
             sum_sq    df      F  PR(>F)
Metodo    1117.7714   2.0  9.215  0.0002
Residual  5276.5085  87.0    NaN     NaN

 ANOVA para Lectura:
             sum_sq    df        F  PR(>F)
Metodo    1451.7091   2.0  13.1348     0.0
Residual  4807.8024  87.0      NaN     NaN

 Corrección de Bonferroni:
   α original = 0.05
   α ajustado = 0.025

   Matemáticas: p = 0.0002 → Significativo
   Lectura: p = 0.0000 → Significativo
```

```
#  Comparaciones múltiples post-hoc (Tukey HSD)
print(" COMPARACIONES MÚLTIPLES POST-HOC (TUKEY HSD) \n")
```

```
# Tukey para Matemáticas
print("Tukey HSD para Matemáticas:")
tukey_mat = pairwise_tukeyhsd(df['Matematicas'], df['Metodo'], alpha=0.05
print(tukey_mat)

print("\n Tukey HSD para Lectura:")
tukey_lec = pairwise_tukeyhsd(df['Lectura'], df['Metodo'], alpha=0.05)
print(tukey_lec)
```

```
 COMPARACIONES MÚLTIPLES POST-HOC (TUKEY HSD)

Tukey HSD para Matemáticas:
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=================================================
group1 group2 meandiff p-adj   lower  upper  reject
-------------------------------------------------
     A      B   8.6083 0.0001  3.8136 13.403   True
     A      C   3.7455 0.1558 -1.0492 8.5402  False
     B      C  -4.8627 0.0461 -9.6574 -0.068   True
-------------------------------------------------

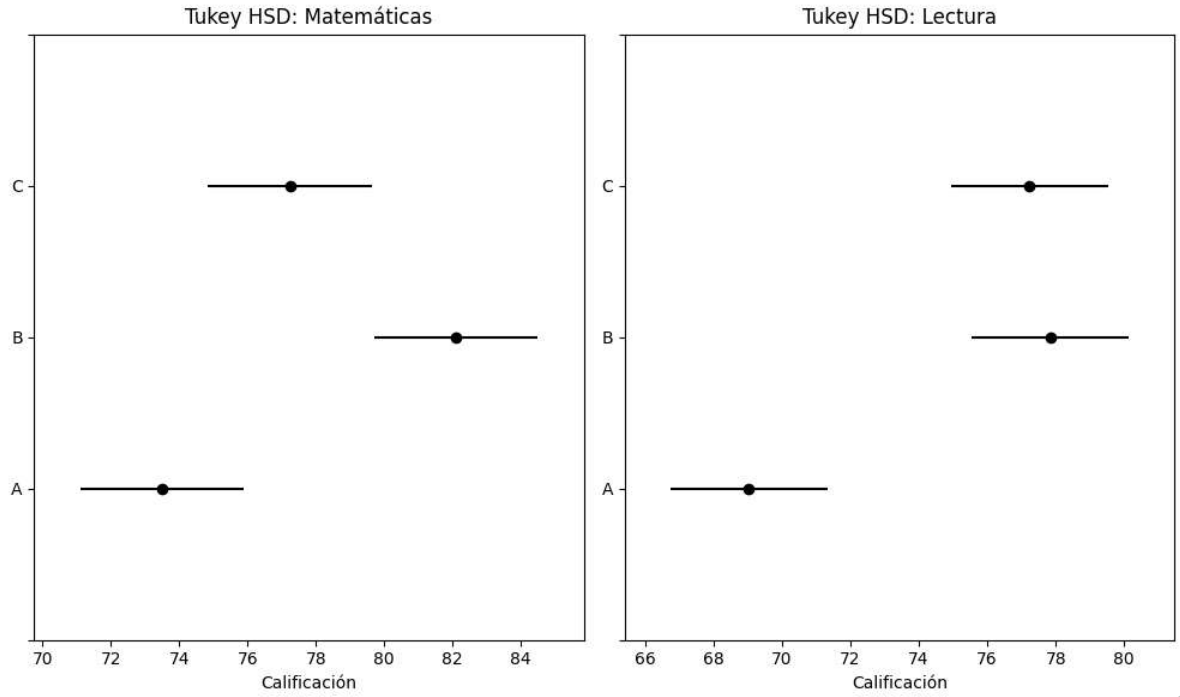
 Tukey HSD para Lectura:
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=================================================
group1 group2 meandiff p-adj   lower   upper  reject
-------------------------------------------------
     A      B   8.8073    0.0  4.2305 13.3841   True
     A      C   8.1996 0.0001  3.6228 12.7764   True
     B      C  -0.6077 0.9463 -5.1845  3.9691  False
-------------------------------------------------
```

```
# Visualización de Tukey HSD
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Tukey Matemáticas
tukey_mat.plot_simultaneous(ax=axes[0])
axes[0].set_title('Tukey HSD: Matemáticas')
axes[0].set_xlabel('Calificación')

# Tukey Lectura
tukey_lec.plot_simultaneous(ax=axes[1])
axes[1].set_title('Tukey HSD: Lectura')
axes[1].set_xlabel('Calificación')

plt.tight_layout()
plt.show()
```




```
# Verificación de supuestos - Normalidad multivariada
print("=== VERIFICACIÓN DE SUPUESTOS ===\n")
print("--- Normalidad Multivariada ---\n")

# Test de Mardia para normalidad multivariada
def mardia_test(data, alpha=0.05):
    """
    Implementación simplificada del test de Mardia
    """
    n, p = data.shape
    mean = np.mean(data, axis=0)
    centered = data - mean
    S = np.cov(data, rowvar=False)
    S_inv = np.linalg.inv(S)

    # Asimetría multivariada
    b1p = 0
    for i in range(n):
        for j in range(n):
            b1p += (centered[i] @ S_inv @ centered[j])**3
    b1p = b1p / (n**2)

    # Curtosis multivariada
    b2p = 0
    for i in range(n):
        b2p += (centered[i] @ S_inv @ centered[i])**2
    b2p = b2p / n

    # Estadísticos de prueba
    skewness_stat = n * b1p / 6
    kurtosis_stat = (b2p - p*(p+2)) / np.sqrt(8*p*(p+2)/n)

    # p-valores
    p_skewness = 1 - stats.chi2.cdf(skewness_stat, p*(p+1)*(p+2)/6)
    p_kurtosis = 2 * (1 - stats.norm.cdf(abs(kurtosis_stat)))
```

```
    return {
        'Skewness statistic': skewness_stat,
        'Skewness p-value': p_skewness,
        'Kurtosis statistic': kurtosis_stat,
        'Kurtosis p-value': p_kurtosis
    }

# Aplicar test por grupo
for metodo in df['Metodo'].unique():
    subset = df[df['Metodo'] == metodo][['Matematicas', 'Lectura']].values
    print(f"Método {metodo}:")
    resultado = mardia_test(subset)
    for key, value in resultado.items():
        print(f"  {key}: {value:.4f}")

    if resultado['Skewness p-value'] > 0.05 and resultado['Kurtosis p-value'
        print("  ✅ Normalidad multivariada aceptable")
    else:
        print("  ⚠️ Posible violación de normalidad multivariada")
    print()
```

```
=== VERIFICACIÓN DE SUPUESTOS ===

--- Normalidad Multivariada ---

Método A:
  Skewness statistic: 1.9333
  Skewness p-value: 0.7480
  Kurtosis statistic: -1.1241
  Kurtosis p-value: 0.2610
  ✅ Normalidad multivariada aceptable

Método B:
  Skewness statistic: 4.0213
  Skewness p-value: 0.4031
  Kurtosis statistic: -0.2271
  Kurtosis p-value: 0.8203
  ✅ Normalidad multivariada aceptable

Método C:
  Skewness statistic: 3.1460
  Skewness p-value: 0.5337
  Kurtosis statistic: -0.9106
  Kurtosis p-value: 0.3625
  ✅ Normalidad multivariada aceptable
```

```
#  Verificación de supuestos - Homogeneidad de covarianzas (Box's M)
print(" Homogeneidad de Matrices de Covarianza (Box's M) \n")

def box_m_test(data, groups):
    """
    Implementación del test M de Box
    """
    group_names = groups.unique()
```

```
    k = len(group_names)
    p = data.shape[1]

    # Tamaños de muestra
    ns = [len(data[groups == g]) for g in group_names]
    n_total = sum(ns)

    # Matrices de covarianza por grupo
    covs = []
    for g in group_names:
        subset = data[groups == g]
        covs.append(np.cov(subset, rowvar=False))

    # Covarianza pooled
    pooled_cov = sum([(n-1) * cov for n, cov in zip(ns, covs)]) / (n_total -

    # Estadístico M
    M = (n_total - k) * np.log(np.linalg.det(pooled_cov))
    for n, cov in zip(ns, covs):
        M -= (n - 1) * np.log(np.linalg.det(cov))

    # Factor de corrección
    c = (2 * p**2 + 3 * p - 1) / (6 * (p + 1) * (k - 1))
    c *= sum([1/(n-1) for n in ns]) - 1/(n_total - k)

    # Estadístico de prueba
    chi2_stat = M * (1 - c)

    # Grados de libertad
    df = p * (p + 1) * (k - 1) / 2

    # p-valor
    p_value = 1 - stats.chi2.cdf(chi2_stat, df)

    return {
        'M statistic': M,
        'Chi-square': chi2_stat,
        'df': df,
        'p-value': p_value
    }

# Aplicar Box's M
resultado_box = box_m_test(df[['Matematicas', 'Lectura']], df['Metodo'])
print("Resultados del test M de Box:")
for key, value in resultado_box.items():
    print(f"  {key}: {value:.4f}")

if resultado_box['p-value'] > 0.05:
    print("\n✅ No se rechaza H₀: Las matrices de covarianza son homogéneas'
else:
    print("\n⚠️ Se rechaza H₀: Posible violación de homogeneidad de covariai
    print("   → Considerar usar Pillai's Trace (más robusto)")
```

Homogeneidad de Matrices de Covarianza (Box's M)

```
Resultados del test M de Box:
  M statistic: 2.3240
  Chi-square: 2.2469
  df: 6.0000
  p-value: 0.8957

✅ No se rechaza H₀: Las matrices de covarianza son homogéneas
```

```
#  Preguntas de análisis
print(" PREGUNTAS DE ANÁLISIS:")
print("\n1. ¿Cuál de los cuatro estadísticos MANOVA es más apropiado y por q
print("2. ¿Qué variable dependiente contribuye más a las diferencias entre g
print("3. ¿Qué pares de grupos son significativamente diferentes?")
print("4. ¿Se cumplen los supuestos del MANOVA? ¿Qué implicaciones tiene?")
print("5. ¿Cómo se relacionan los resultados del LDA con los del MANOVA?")
```

```
 PREGUNTAS DE ANÁLISIS:

1. ¿Cuál de los cuatro estadísticos MANOVA es más apropiado y por qué?
2. ¿Qué variable dependiente contribuye más a las diferencias entre grupos?
3. ¿Qué pares de grupos son significativamente diferentes?
4. ¿Se cumplen los supuestos del MANOVA? ¿Qué implicaciones tiene?
5. ¿Cómo se relacionan los resultados del LDA con los del MANOVA?
```

Escribe tus respuestas aquí:

1. En este caso los cuatro estad´siticos condcuerdan en los resultados entonces la conclusión no se guía meramente por uno solo aunque el Pillai's trace es el más robusto ante violaciones de normalidad o de homogeneidad de cov.
2. La varaible dependiente que contribuye más al discriminante entre los grupos es la de lectura ya que su F es más grande que la de mátematicas al igual que n^2. El método rinde mucho menos que los otros dos en lectura, mientras que en matemátcias solo B se diferencia claramente de los demás.
3. En matemáticas los pares A y B además de el B con C si difieron, mientras que en lectura difieron A y B al igual que A con C
4. Sí se cumplen los supuestos. Mardia p>0.05 en asimetría y curtosis y Box's Mda p=0.896. lo que implica que los resultados del MANOVA son confiables y los cuatro estadísticos sirven
5. Ambos utilizan la misma matriz y sus eigen valores son la base de los cuatro estadísticos, pero LDA usa los vectores propios de esa misma matriz como ejes discriminantes, y sus coeficientes muestran qué variable pesa más en la separación.

Actividad MANOVA

Ahora repite todo para el siguiente dataset de load_iris

```
# Celda 3: Cargar el dataset Iris (datos reales)
# El dataset Iris contiene mediciones de 3 especies de flores
iris = load_iris()

# Crear DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['Especie'] = iris.target_names[iris.target]

# Renombrar columnas para claridad
df.columns = ['Longitud_Sepalo', 'Ancho_Sepalo',
              'Longitud_Petalo', 'Ancho_Petalo', 'Especie']

print(f"Forma del dataset: {df.shape}")
print(f"Especies: {df['Especie'].unique()}")
print(f"Observaciones por especie:")
print(df['Especie'].value_counts())
df.head(10)
```

```
Forma del dataset: (150, 5)
Especies: ['setosa' 'versicolor' 'virginica']
Observaciones por especie:
Especie
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

	Longitud_Sepalo	Ancho_Sepalo	Longitud_Petalo	Ancho_Petalo	Especie
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
5	5.4	3.9	1.7	0.4	setosa
6	4.6	3.4	1.4	0.3	setosa
7	5.0	3.4	1.5	0.2	setosa
8	4.4	2.9	1.4	0.2	setosa
9	4.9	3.1	1.5	0.1	setosa

Pasos siguientes: Generar código con df | New interactive sheet

```
# Estadísticas descriptivas por especie
print("ESTADÍSTICAS DESCRIPTIVAS POR ESPECIE: \n")

variables = ['Longitud_Sepalo', 'Ancho_Sepalo', 'Longitud_Petalo', 'Ancho_Pe

estadisticas = df.groupby('Especie')[variables].agg(['mean', 'std', 'min', '


for var in variables:
    print(f"--- {var} ---")
    print(estadisticas[var])
    print()
```

```
ESTADÍSTICAS DESCRIPTIVAS POR ESPECIE:

--- Longitud_Sepalo ---
             mean    std  min  max
Especie
setosa      5.006  0.352  4.3  5.8
versicolor  5.936  0.516  4.9  7.0
virginica   6.588  0.636  4.9  7.9

--- Ancho_Sepalo ---
             mean    std  min  max
Especie
setosa      3.428  0.379  2.3  4.4
versicolor  2.770  0.314  2.0  3.4
virginica   2.974  0.322  2.2  3.8

--- Longitud_Petalo ---
             mean    std  min  max
Especie
setosa      1.462  0.174  1.0  1.9
versicolor  4.260  0.470  3.0  5.1
virginica   5.552  0.552  4.5  6.9

--- Ancho_Petalo ---
             mean    std  min  max
Especie
setosa      0.246  0.105  0.1  0.6
versicolor  1.326  0.198  1.0  1.8
virginica   2.026  0.275  1.4  2.5
```

```
# Visualización
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Boxplots de las 4 variables
sns.boxplot(x='Especie', y='Longitud_Sepalo', data=df, ax=axes[0, 0], palett
axes[0, 0].set_title('Longitud del Sépalo')

sns.boxplot(x='Especie', y='Ancho_Sepalo', data=df, ax=axes[0, 1], palette='
axes[0, 1].set_title('Ancho del Sépalo')

sns.boxplot(x='Especie', y='Longitud_Petalo', data=df, ax=axes[1, 0], palett
axes[1, 0].set_title('Longitud del Pétalo')
```

```
sns.boxplot(x='Especie', y='Ancho_Petalo', data=df, ax=axes[1, 1], palette='
axes[1, 1].set_title('Ancho del Pétalo')

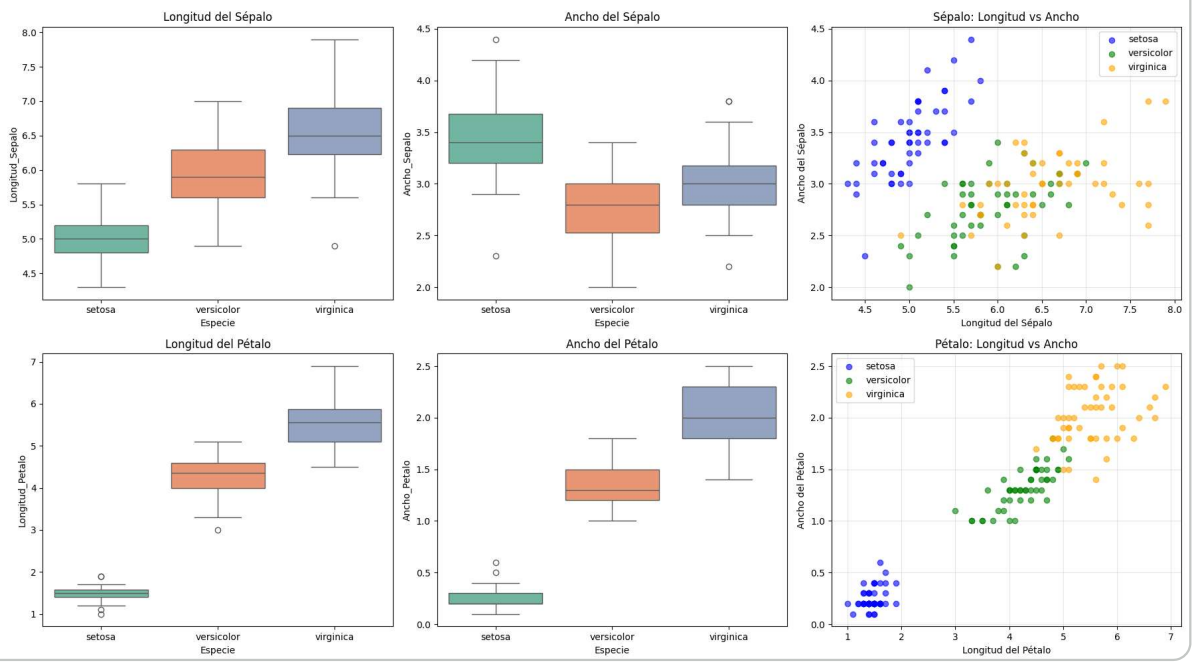
# Scatter plots
colors = {'setosa': 'blue', 'versicolor': 'green', 'virginica': 'orange'}

for especie in df['Especie'].unique():
    subset = df[df['Especie'] == especie]
    axes[0, 2].scatter(subset['Longitud_Sepalo'], subset['Ancho_Sepalo'],
                       alpha=0.6, label=especie, color=colors[especie])
    axes[1, 2].scatter(subset['Longitud_Petalo'], subset['Ancho_Petalo'],
                       alpha=0.6, label=especie, color=colors[especie])

axes[0, 2].set_xlabel('Longitud del Sépalo')
axes[0, 2].set_ylabel('Ancho del Sépalo')
axes[0, 2].set_title('Sépalo: Longitud vs Ancho')
axes[0, 2].legend()
axes[0, 2].grid(True, alpha=0.3)

axes[1, 2].set_xlabel('Longitud del Pétalo')
axes[1, 2].set_ylabel('Ancho del Pétalo')
axes[1, 2].set_title('Pétalo: Longitud vs Ancho')
axes[1, 2].legend()
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```




```
# Matriz de correlación
variables = ['Longitud_Sepalo', 'Ancho_Sepalo', 'Longitud_Petalo', 'Ancho_Pe


print("correlació total")
print(df[variables].corr().round(4))
```

```
#por grupos
centrado = df[variables] - df.groupby('Especie')[variables].transform('mean'
print("correlacipn dentro de grupos")
print(centrado.corr().round(4))

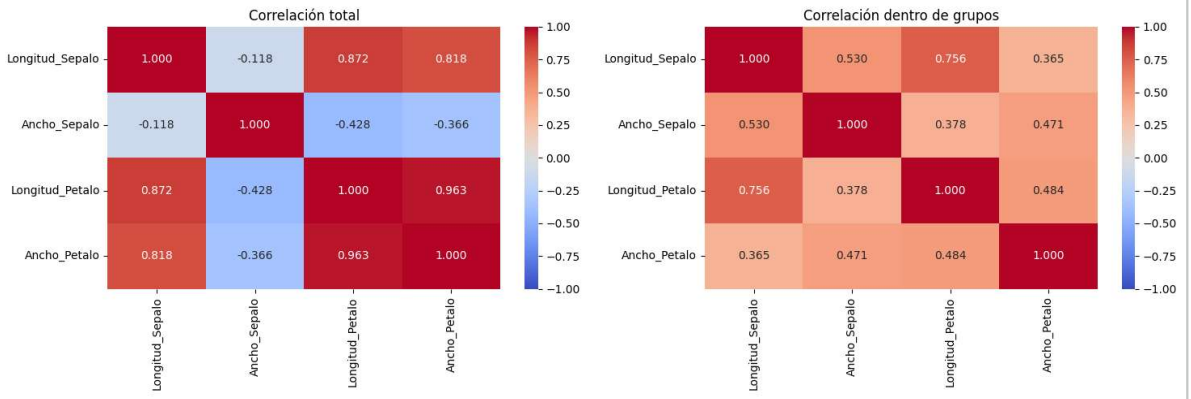
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.heatmap(df[variables].corr(), annot=True, cmap='coolwarm', center=0,
            vmin=-1, vmax=1, fmt='.3f', ax=axes[0])
axes[0].set_title('Correlación total')

sns.heatmap(centrado.corr(), annot=True, cmap='coolwarm', center=0,
            vmin=-1, vmax=1, fmt='.3f', ax=axes[1])
axes[1].set_title('Correlación dentro de grupos')

plt.tight_layout()
plt.show()
```

```
correlació total
                Longitud_Sepalo  Ancho_Sepalo  Longitud_Petalo  Ancho_Petalo
Longitud_Sepalo           1.0000       -0.1176           0.8718        0.8179
Ancho_Sepalo             -0.1176        1.0000          -0.4284       -0.3661
Longitud_Petalo           0.8718       -0.4284           1.0000        0.9629
Ancho_Petalo              0.8179       -0.3661           0.9629        1.0000
correlacipn dentro de grupos
                Longitud_Sepalo  Ancho_Sepalo  Longitud_Petalo  Ancho_Petalo
Longitud_Sepalo           1.0000        0.5302           0.7562        0.3645
Ancho_Sepalo              0.5302        1.0000           0.3779        0.4705
Longitud_Petalo           0.7562        0.3779           1.0000        0.4845
Ancho_Petalo              0.3645        0.4705           0.4845        1.0000
```




```
print("MANOVA para datset IRIS")

manova_iris = MANOVA.from_formula(
    'Longitud_Sepalo + Ancho_Sepalo + Longitud_Petalo + Ancho_Petalo ~ Espec
    data=df
)
resultado_iris = manova_iris.mv_test()

print(resultado_iris)
```

```
MANOVA para datset IRIS
                  Multivariate linear model
================================================================
```

```
------------------------------------------------------------------
       Intercept         Value  Num DF  Den DF   F Value  Pr > F
------------------------------------------------------------------
          Wilks' lambda  0.0170 4.0000 144.0000 2086.7720 0.0000
         Pillai's trace  0.9830 4.0000 144.0000 2086.7720 0.0000
 Hotelling-Lawley trace 57.9659 4.0000 144.0000 2086.7720 0.0000
    Roy's greatest root 57.9659 4.0000 144.0000 2086.7720 0.0000
------------------------------------------------------------------

------------------------------------------------------------------
        Especie          Value  Num DF  Den DF   F Value  Pr > F
------------------------------------------------------------------
          Wilks' lambda  0.0234 8.0000 288.0000  199.1453 0.0000
         Pillai's trace  1.1919 8.0000 290.0000   53.4665 0.0000
 Hotelling-Lawley trace 32.4773 8.0000 203.4024  582.1970 0.0000
    Roy's greatest root 32.1919 4.0000 145.0000 1166.9574 0.0000
==================================================================
```

```
print("ANOVA univariado")

variables = ['Longitud_Sepalo', 'Ancho_Sepalo', 'Longitud_Petalo', 'Ancho_Pe

alpha = 0.05
alpha_bonferroni = alpha / len(variables)

resumen = []

for var in variables:
    modelo = ols(f'{var} ~ Especie', data=df).fit()
    tabla = sm.stats.anova_lm(modelo, typ=2)

    print(f"ANOVA para {var}:")
    print(tabla.round(4))
    print()

    F = tabla.loc['Especie', 'F']
    p = tabla.loc['Especie', 'PR(>F)']
    eta2 = tabla.loc['Especie', 'sum_sq'] / tabla['sum_sq'].sum()

    resumen.append({
        'Variable': var,
        'F': round(F, 2),
        'p-valor': p,
        'eta²': round(eta2, 3),
        'Significativo (Bonferroni)': 'Sí' if p < alpha_bonferroni else 'No'
    })

print("Corrección de Bonferroni:")
print(f"   α original = {alpha}")
print(f"   α ajustado = {alpha_bonferroni}\n")

print("RESUMEN (ordenado por tamaño del efecto):")
print(pd.DataFrame(resumen).sort_values('eta²', ascending=False).to_string(i
```

```
ANOVA univariado
ANOVA para Longitud_Sepalo:
           sum_sq     df         F  PR(>F)
Especie   63.2121    2.0  119.2645     0.0
Residual  38.9562  147.0       NaN     NaN

ANOVA para Ancho_Sepalo:
           sum_sq     df      F  PR(>F)
Especie   11.3449    2.0  49.16     0.0
Residual  16.9620  147.0    NaN     NaN

ANOVA para Longitud_Petalo:
            sum_sq     df          F  PR(>F)
Especie   437.1028    2.0  1180.1612     0.0
Residual   27.2226  147.0        NaN     NaN

ANOVA para Ancho_Petalo:
           sum_sq     df         F  PR(>F)
Especie   80.4133    2.0  960.0071     0.0
Residual   6.1566  147.0       NaN     NaN

Corrección de Bonferroni:
   α original = 0.05
   α ajustado = 0.0125

RESUMEN (ordenado por tamaño del efecto):
       Variable       F      p-valor  eta² Significativo (Bonferroni)
Longitud_Petalo 1180.16 2.856777e-91 0.941                         Sí
   Ancho_Petalo  960.01 4.169446e-85 0.929                         Sí
Longitud_Sepalo  119.26 1.669669e-31 0.619                         Sí
   Ancho_Sepalo   49.16 4.492017e-17 0.401                         Sí
```

```
print("comparaciones multiples post hoc")

variables = ['Longitud_Sepalo', 'Ancho_Sepalo', 'Longitud_Petalo', 'Ancho_Pe
tukey_resultados = {}

for var in variables:
    print(f"Tukey HSD para {var}:")
    tukey = pairwise_tukeyhsd(df[var], df['Especie'], alpha=0.05)
    tukey_resultados[var] = tukey
    print(tukey)
    print()

# Visualización de los intervalos de confianza
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

for ax, var in zip(axes.flatten(), variables):
    tukey_resultados[var].plot_simultaneous(ax=ax)
    ax.set_title(f'Tukey HSD: {var}')
    ax.set_xlabel('Valor medio')

plt.tight_layout()
plt.show()
```

comparaciones multiples post hoc

```
#verificar supuestos
variables = ['Longitud_Sepalo', 'Ancho_Sepalo', 'Longitud_Petalo', 'Ancho

#mardia
def mardia_test(data, alpha=0.05):
    n, p = data.shape
    mean = np.mean(data, axis=0)
    centered = data - mean
    S = np.cov(data, rowvar=False)
    S_inv = np.linalg.inv(S)


    b1p = 0
    for i in range(n):
        for j in range(n):
            b1p += (centered[i] @ S_inv @ centered[j])**3
    b1p = b1p / (n**2)


    b2p = 0
    for i in range(n):
        b2p += (centered[i] @ S_inv @ centered[i])**2
    b2p = b2p / n

    skewness_stat = n * b1p / 6
    kurtosis_stat = (b2p - p*(p+2)) / np.sqrt(8*p*(p+2)/n)

    p_skewness = 1 - stats.chi2.cdf(skewness_stat, p*(p+1)*(p+2)/6)
    p_kurtosis = 2 * (1 - stats.norm.cdf(abs(kurtosis_stat)))

    return {
        'Skewness statistic': skewness_stat,
        'Skewness p-value': p_skewness,
        'Kurtosis statistic': kurtosis_stat,
        'Kurtosis p-value': p_kurtosis
    }

print("verificacion de supuestos")
print("normalidad multivariada")

for especie in df['Especie'].unique():
    subset = df[df['Especie'] == especie][variables].values
    print(f"Especie {especie}:")
    resultado = mardia_test(subset)
    for key, value in resultado.items():
        print(f"  {key}: {value:.4f}")

    if resultado['Skewness p-value'] > 0.05 and resultado['Kurtosis p-val
        print("Normalidad multivariada aceptable")
    else:
        print("Posible violación de normalidad multivariada")
    print()
```

```
def box_m_test(data, groups):
    group_names = groups.unique()
    k = len(group_names)
    p = data.shape[1]

    ns = [len(data[groups == g]) for g in group_names]
    n_total = sum(ns)

    covs = [np.cov(data[groups == g], rowvar=False) for g in group_names]

    pooled_cov = sum([(n-1) * cov for n, cov in zip(ns, covs)]) / (n_tota

    M = (n_total - k) * np.log(np.linalg.det(pooled_cov))
    for n, cov in zip(ns, covs):
        M -= (n - 1) * np.log(np.linalg.det(cov))

    c = (2 * p**2 + 3 * p - 1) / (6 * (p + 1) * (k - 1))
    c *= sum([1/(n-1) for n in ns]) - 1/(n_total - k)

    chi2_stat = M * (1 - c)
    gl = p * (p + 1) * (k - 1) / 2
    p_value = 1 - stats.chi2.cdf(chi2_stat, gl)

    return {'M statistic': M, 'Chi-square': chi2_stat, 'df': gl, 'p-value

print(" Homogeneidad de Covarianzas ")
resultado_box = box_m_test(df[variables], df['Especie'])
for key, value in resultado_box.items():
    print(f"  {key}: {value:.4f}")

if resultado_box['p-value'] > 0.05:
    print("No se rechaza la hipotesis nula: Las matrices de covarianza so
else:
    print("Se rechaza hipoterisi nula: Posible falta de homogeneidad de c
```

```
verificacion de supuestos
normalidad multivariada
Especie setosa:
  Skewness statistic: 24.1551
  Skewness p-value: 0.2357
  Kurtosis statistic: 0.7587
  Kurtosis p-value: 0.4480
Normalidad multivariada aceptable

Especie versicolor:
  Skewness statistic: 23.7039
  Skewness p-value: 0.2556
  Kurtosis statistic: -1.0342
  Kurtosis p-value: 0.3010
Normalidad multivariada aceptable

Especie virginica:
  Skewness statistic: 24.7257
  Skewness p-value: 0.2121
  Kurtosis statistic: -0.3384
  Kurtosis p-value: 0.7350
```

```
Normalidad multivariada aceptable

 Homogeneidad de Covarianzas
  M statistic: 146.6632
  Chi-square: 140.9430
  df: 20.0000
  p-value: 0.0000
Se rechaza hipoterisi nula: Posible falta de homogeneidad de covarianzas
```